DATAMATIKER · 2. SEMESTER · PROJEKTRAPPORT

Cupcake

Webshop til Olsker Cupcakes



Navn	Studie-email	GitHub
Daud Ibragimov	cph-di36@stud.ek.dk	daud0095
Feyzullah Goc	fego1001@stud.ek.dk	feyzullahgoctr
Cezary Rzempoluch	Cerz1000@stud.ek.dk	extile-crypto

Gruppe	C
Tidspunkt	Efterår 2026, afleveret 11. november 2026
GitHub repository	https://github.com/Feyzullahgoctr/Cupcake-Group-C
YouTube-video	https://www.youtube.com/watch?v=QFPe-SdEnEM Kort video om vores projekt

Indholdsfortegnelse

1 Indledning 3

Baggrund 3

Teknologivalg 4

2 Krav 4

3 Domænemodel og ER-diagram 7

4 Mockups 9

5 Navigationsdiagram 10

Fælles navigationsmenu 10

Adgang til siderne 10

6 Beskrivelse af brugerflowet 11

Gæst (ikke logget ind) 11

Hvis brugeren ikke er admin (kunde) 11

Hvis brugeren er admin 11

7 Implementation 12

Sammenhæng mellem controller, route, HTML, service og mapper 12

Routes og deres vej gennem systemet 13

8 Test 14

9 Status på implementation 16

10 Proces 16

1 Indledning

Denne rapport beskriver, hvordan vi har lavet en webshop til bageriet Olsker Cupcakes på Bornholm. I webshoppen kan kunder sammensætte cupcakes, betale med en saldo på deres konto og se deres ordrer. Butikkens administrator kan se alle ordrer og kunder og slette ordrer.

Rapporten er skrevet til en anden datamatikerstuderende på 2. semester, som ikke kender Cupcake-opgaven. Læseren kender Java og SQL, men ikke vores system.

Rapporten starter med baggrund og krav. Derefter kommer domænemodel, database, mockups og navigation. Til sidst beskriver vi udvalgte dele af koden, vores test, hvor langt vi er nået, og hvordan vi har arbejdet sammen.

Baggrund

Olsker Cupcakes er et lille bageri på Bornholm, som laver cupcakes. Bageriet vil gerne sælge cupcakes over nettet, så kunderne kan bestille hjemmefra og hente dem i butikken.

Kunden har disse krav til systemet, forklaret bredt:

- Kunden skal kunne bestille en eller flere cupcakes, hvor man selv vælger bund og top.
- Kunden skal kunne oprette en konto og logge ind.
- Kunden betaler med penge, som står på kontoen (en saldo), og skal kunne se sin saldo.
- Kunden skal kunne se sin kurv og fjerne ting fra den, før der betales.
- Ejeren (admin) skal kunne se alle ordrer og alle kunder.
- Ejeren skal kunne slette en ordre, fx hvis den aldrig bliver betalt eller hentet.

Teknologivalg

Systemet er en Java-webapplikation, som kører på port 7070 og gemmer data i PostgreSQL. Versionerne fra `pom.xml` er:

Teknologi	Version	Bruges til
Java	21	Programmeringssprog
Javalin	7.2.3	Webserver og routes
Thymeleaf (javalin-rendering-thymeleaf)	7.2.3	HTML-skabeloner
PostgreSQL JDBC-driver	42.7.8	Forbindelse fra Java til databasen
HikariCP	7.0.2	Connection pool
SLF4J (slf4j-simple)	2.0.17	Logning af fejl
JUnit Jupiter	RELEASE (nyeste)	Test
Maven	3.9.9	Bygning og afhængigheder
IntelliJ IDEA	2026.3	Udviklingsmiljø
pgAdmin 4	9.18	Database-værktøj og ERD
Figma	–	Mockups
GitHub	–	Kode, pull requests og GitHub Projects

2 Krav

> **Vision:** Olsker Cupcakes vil nå flere kunder ved at sælge over nettet. Kunderne kan bestille, når det passer dem, og ejeren får et samlet overblik over ordrer og kunder. Det sparer tid i butikken, fordi ordren allerede er betalt, når kunden kommer.

User stories: Vi har lavet alle 8 user stories. Saldo (indsæt penge og se saldo) er lavet som en del af US-1, fordi kunden skal have penge på kontoen for at kunne betale.

US-1 Som kunde kan jeg bestille og betale cupcakes med en valgfri bund og top, sådan at jeg senere kan køre forbi butikken og hente min ordre uden at stå i kø.

ACCEPTKRITERIER

- Givet at jeg er logget ind, når jeg vælger en valgfri bund og top til mine cupcakes, så kan jeg tilføje dem til min ordre.
- Givet at jeg har gennemgået min indkøbskurv, når jeg går videre til betaling, så kan jeg betale for min ordre.
- Givet at min ordre er betalt, så får jeg en ordrebekræftelse med information om afhentning.

US-2 Som kunde kan jeg oprette en konto/profil for at kunne betale og gemme en ordre.

ACCEPTKRITERIER

- Givet at jeg ikke allerede har en konto, når jeg indtaster mine brugeroplysninger og opretter en profil, så bliver min konto oprettet.
- Givet at jeg indtaster en emailadresse, der allerede er registreret, så får jeg vist en fejlmeddelelse.
- Givet at jeg opretter en konto med gyldige oplysninger, så kan jeg logge ind med min email og adgangskode.

US-3 Som kunde kan jeg se mine valgte ordrelinjer i en indkøbskurv, så jeg kan se den samlede pris.

ACCEPTKRITERIER

- Givet at jeg har valgt en eller flere cupcakes, når jeg åbner indkøbskurven, så kan jeg se alle mine valgte ordrelinjer.
- Givet at jeg har tilføjet cupcakes til indkøbskurven, så vises prisen for hver ordrelinje.
- Givet at jeg har flere ordrelinjer i indkøbskurven, så vises den samlede pris for hele ordren.
- Givet at min indkøbskurv er tom, så vises en besked om, at der ikke er nogen varer i kurven.

US-4 Som kunde eller administrator kan jeg logge på systemet med email og kodeord. Når jeg er logget på, skal jeg kunne se min email på hver side (evt. i topmenuen, som vist på mockup'en).

ACCEPTKRITERIER

- Givet at jeg er en registreret kunde eller administrator, når jeg indtaster min email og adgangskode korrekt, så bliver jeg logget ind på systemet.
- Givet at jeg indtaster en forkert email eller adgangskode, når jeg forsøger at logge ind, så får jeg vist en fejlmeddelelse.
- Givet at jeg er logget ind, så kan jeg se min emailadresse på alle sider i systemet.

US-5 Som administrator kan jeg se alle ordrer i systemet, så jeg kan se, hvad der er blevet bestilt.

ACCEPTKRITERIER

- Givet at jeg er logget ind som administrator, når jeg åbner ordreoversigten, så kan jeg se alle ordrer i systemet.
- Givet at der findes ordrer i systemet, så kan jeg se relevante oplysninger om hver ordre, herunder kunden, ordrelinjer, samlet pris og betalingsstatus.

US-6 Som administrator kan jeg se alle kunder i systemet og deres ordrer, sådan at jeg kan følge op på ordrer og holde styr på mine kunder.

ACCEPTKRITERIER

- Givet at jeg er logget ind som administrator, når jeg åbner kundeoversigten, så kan jeg se alle registrerede kunder.
- Givet at jeg vælger en bestemt kunde, så kan jeg se kundens oplysninger.
- Givet at den valgte kunde har afgivet ordrer, så kan jeg se kundens ordrer.

US-7 Som kunde kan jeg fjerne en ordrelinje fra min indkøbskurv, så jeg kan justere min ordre.

ACCEPTKRITERIER

- Givet at jeg har en eller flere ordrelinjer i min indkøbskurv, når jeg vælger at fjerne en ordrelinje, så bliver den fjernet fra indkøbskurven.
- Givet at jeg har fjernet en ordrelinje, så bliver den samlede pris automatisk opdateret.

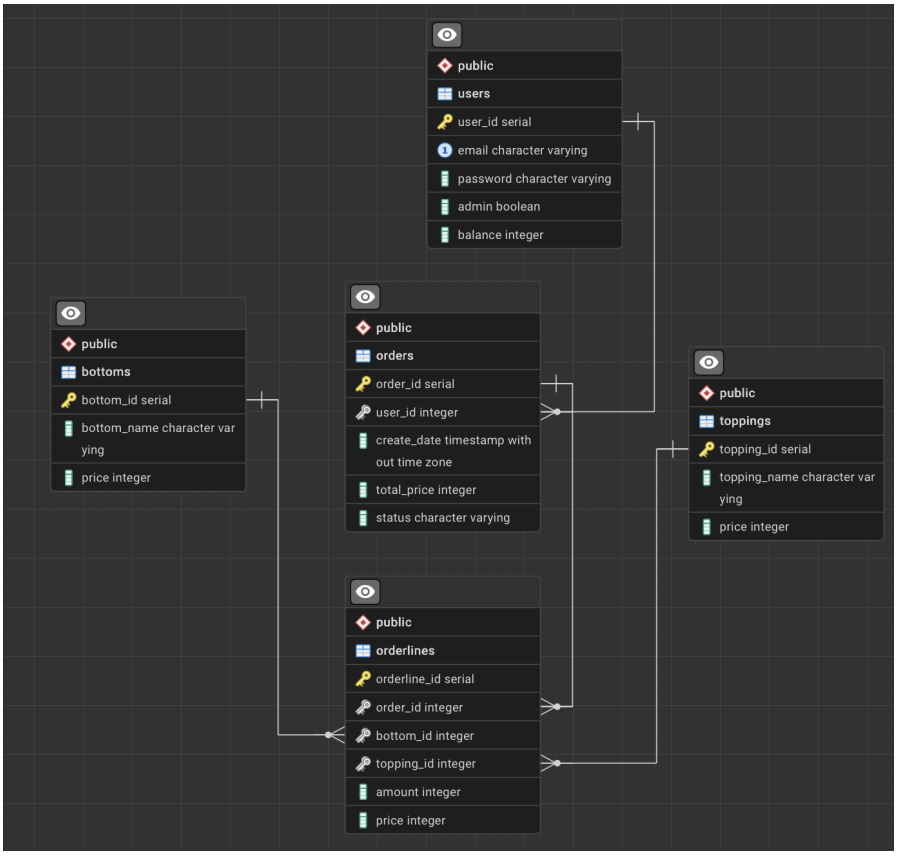
US-8 Som administrator kan jeg fjerne en ordre, så systemet ikke kommer til at indeholde ugyldige ordrer. F.eks. hvis kunden aldrig har betalt.

ACCEPTKRITERIER

- Givet at jeg er logget ind som administrator, når jeg vælger at slette en ordre, så bliver ordren fjernet fra systemet.
- Givet at en ordre er slettet, så bliver dens ordrelinjer også slettet, og ordren vises ikke længere i ordreoversigten.

3 Domænemodel og ER-diagram

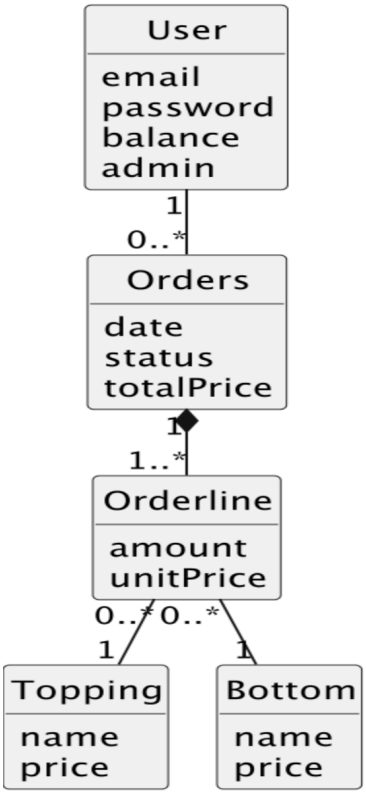
ER-diagram



Figur 1: ER-diagram

Fra	Til	Type	Forklaring
users	orders	1 til mange	En bruger kan have mange ordrer, men en ordre hører til én bruger
orders	orderlines	1 til mange	En ordre består af en eller flere ordrelinjer
bottoms	orderlines	1 til mange	Samme bund kan indgå i mange ordrelinjer
toppings	orderlines	1 til mange	Samme topping kan indgå i mange ordrelinjer

Domænemodel



Figur 2: Domænemodel

I vores domænemodel har vi fem begreber: **User**, **Orders**, **Orderline**, **Bottom** og **Topping**:

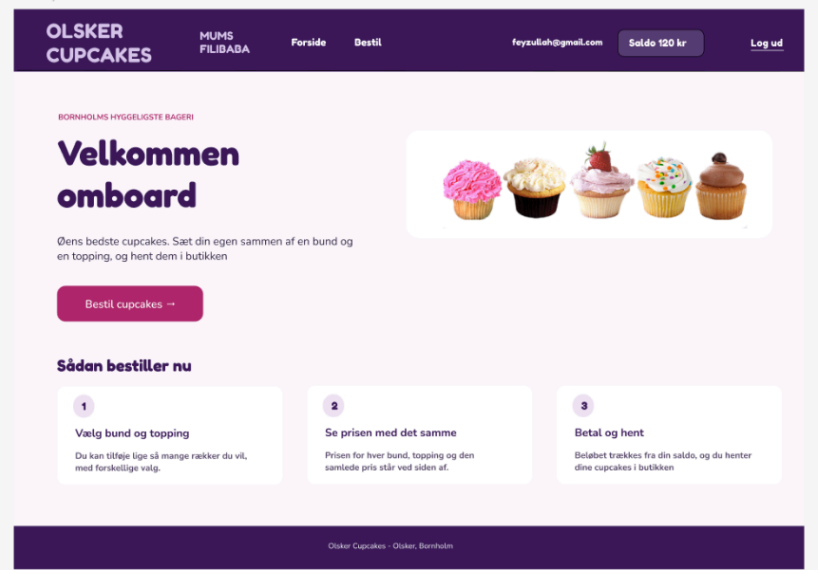
- En User kan have 0 eller flere ordrer. En ny bruger har ingen ordrer endnu.
- En ordre består af 1 eller flere ordrelinjer. En ordrelinje findes ikke uden sin ordre, som også kaldes for komposition.
- En ordrelinje har præcis en bund og en top. En bund og top kan være i mange ordrelinjer.

Den relation, vi har overvejet mest, er ordrelinjen. Man kunne tænke, at en ordre bare har en liste af bunde og toppe. Men så ved man ikke, hvilken bund der hører til hvilken top, eller hvor mange der er af hver. Derfor er ordrelinjen sit eget begreb med antal og pris.

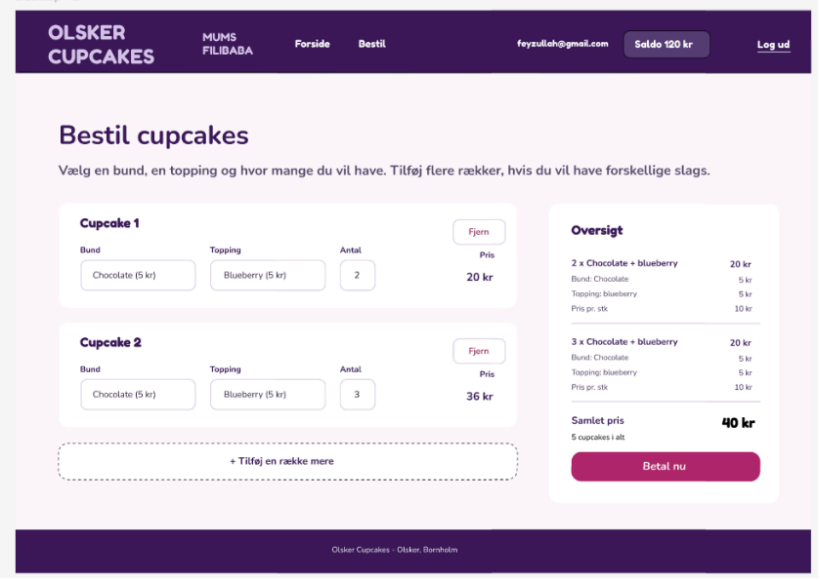
Admin er ikke sin egen klasse. En admin er en bruger, hvor admin er sand. Vi valgte det, fordi admin og kunde logger ind på samme måde.

4 Mockups

Vi lavede hi-fi mockups i Figma, før vi byggede siderne i HTML, CSS og Thymeleaf. På den måde kunne hele gruppen se og blive enige om designet, før vi begyndte at kode.



Figur 3: Forsiden i Figma



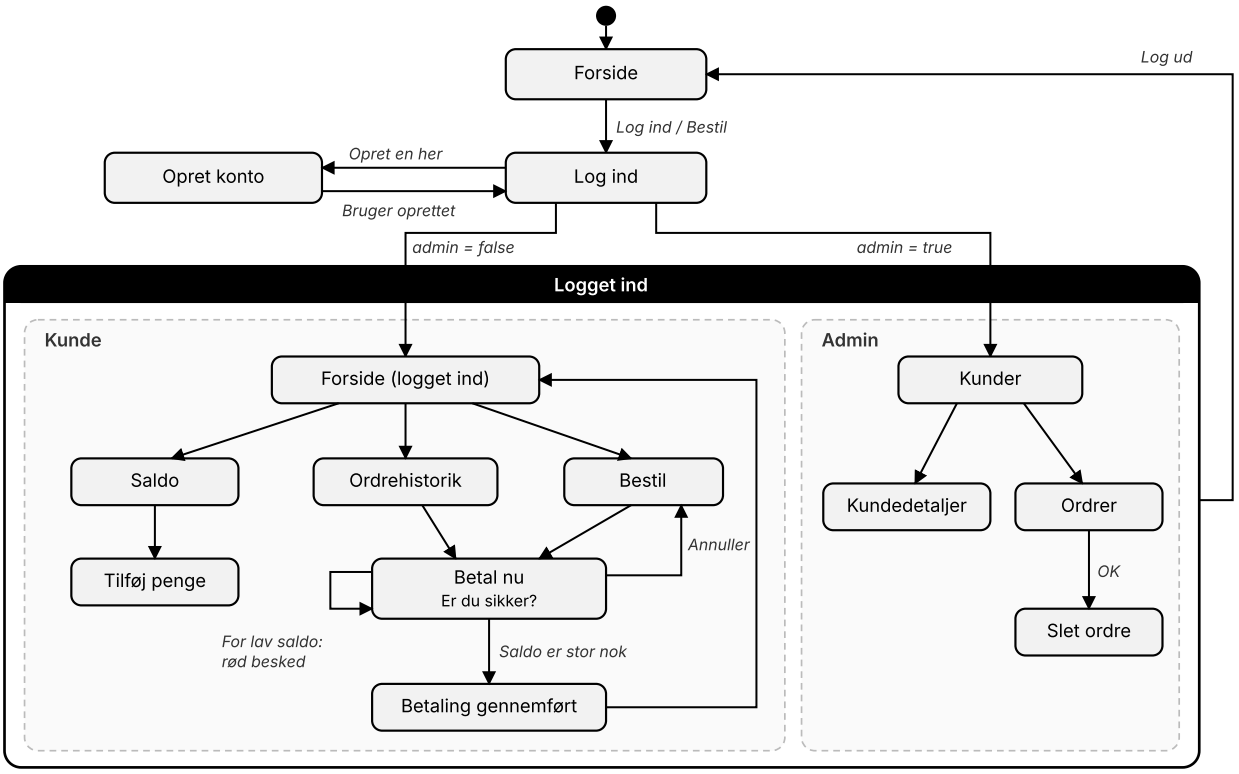
Figur 4: Bestillingssiden i Figma med rækker til venstre og oversigt til højre

Den færdige side ligner mockuppen meget. Forskellene er:

- Saldoen i topmenuen blev en dropdown, hvor man kan indsætte 50, 100 eller 250 kr.
- Topmenuen fik også links til Ordrehistorik og Kurv.
- Bestillingssiden fik en "Redigere"-knap, så man kan ændre kurven, efter man har trykket "Læg i kurv".

5 Navigationsdiagram

Navigationsdiagrammet viser, hvilke sider brugeren kan se, og hvordan man kommer fra den ene side til den anden.



Figur 5: Navigationsdiagram

Fælles navigationsmenu

Alle kundesider har den samme navigationsmenu øverst på siden.

- Menuen indeholder "Forside", "Ordrehistorik", "Kurv" og enten "Log ind" eller "Log ud".
- Når brugeren er logget ind, vises brugerens e-mailadresse og saldo også. Derudover er der mulighed for at indsætte penge på kontoen.
- Admin-siderne har deres egen navigationsmenu med "Ordrer", "Kunder" og "Log ud".

Adgang til siderne

- Forsiden, login-siden og siden "Opret konto" er tilgængelige for alle brugere.
- "Kurv", "Ordrehistorik" og "Bestil" er kun tilgængelige for brugere, der er logget ind.
- "Kunder", "Kundedetaljer" og "Ordrer" er kun tilgængelige for administratorer.

6 Beskrivelse af brugerflowet

Gæst (ikke logget ind)

- Først kan man se forsiden uden at være logget ind.
- Forsiden har en "Bestil"-knap og en "Log ind"-knap.
- Hvis man trykker på "Bestil" uden at være logget ind, sendes man til loginsiden.
- Her kan man oprette en konto eller logge ind.
- Systemet har nogle kontroller, for eksempel:
  - Den samme e-mail kan ikke registreres to gange.
  - Adgangskoden kontrolleres mod databasen ved login.

Hvis brugeren ikke er admin (kunde)

- Når man er logget ind, kan man se sin saldo og sin ordrehistorik og bestille cupcakes.
- **Saldo:** Kunden kan sætte penge ind ved at vælge 50, 100 eller 250 kr. i en dropdown, eller fjerne alle pengene. Saldoen er en simpel kontobalance.
- **Ordrehistorik:** Her kan kunden se alle sine tidligere ordrer. Hvis en ordre ikke er betalt, kan kunden betale den herfra med "Betal nu" eller slette den med "Slet ordre".
- **Bestil:** Kunden vælger bund, topping og antal for hver cupcake, tilføjer flere rækker efter behov og trykker "Læg i kurv".
  - Ordren gemmes i databasen med status "unpaid".
  - Hvis kunden ikke betaler, men går til forsiden og kommer tilbage til bestillingssiden, kan kunden se den samme ordre igen, fordi den stadig står som "unpaid".
  - Kurven huskes via `order_id` i sessionen. Logger kunden ud, findes den ubetalte ordre i ordrehistorikken, hvor den kan betales.
- Efter "Læg i kurv" vises en oversigt til højre med hver linje, antal cupcakes og samlet pris. Kunden kan trykke "Redigere" og ændre kurven.
- Kunden trykker "Betal nu", og siden "Er du sikker?" vises. Ved bekræftelse tjekkes saldoen.
  - Hvis saldoen er for lav, vises en rød besked.
  - Ellers får ordren status "Paid", saldoen trækkes, og siden "Betaling gennemført" vises.

Hvis brugeren er admin

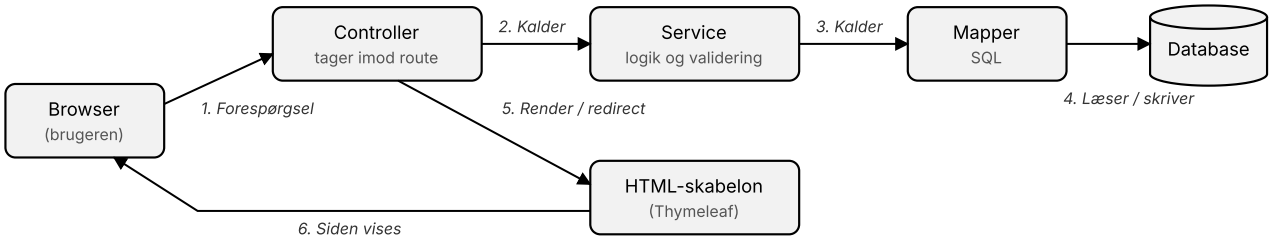
- Admin logger ind på den samme loginside og sendes til kundeoversigten.
- Admin kan se alle kunder og deres ordrer.
- Admin kan se alle ordrer og slette kunders ordrer.

7 Implementation

Sammenhæng mellem controller, route, HTML, service og mapper

- **Controlleren** modtager forespørgslen og kalder en service.
- **Servicen** indeholder logik og validering og kalder en mapper, som er den eneste klasse, der taler med databasen.
- Til sidst sender controlleren data til en HTML-skabelon (Thymeleaf) eller sender brugeren videre med et redirect.

Fordelen er, at hvert lag har et ansvar. Controlleren kender hverken SQL eller databasen, og mapperen kender ikke HTTP. Det gør koden lettere at ændre og teste.



Figur 6: Vejen fra browser til database og tilbage

Ordforklaring

Route	En adresse i browseren, fx `/login`. GET henter en side, POST sender data fra en formular.
Session	Serveren husker om brugeren mellem siderne, fx hvem der er logget ind.
Render	Controlleren viser en HTML-skabelon med data.
Redirect	Controlleren sender browseren videre til en anden route.

Routes og deres vej gennem systemet

Tabellen viser, hvad der sker for hver route.

Controller	Route	Hvad sker der?	Resultat
UserController	`GET /`	Viser forsiden	index.html
	`GET /login`	Viser loginformularen	login.html
	`POST /login`	Tjekker email og adgangskode og logger brugeren ind	Admin sendes til /admin/kunder. Kunde sendes til /. Fejl: login.html med besked
	`GET /logout`	Logger brugeren ud	Sendes til /
	`GET /register`	Viser formularen til at oprette en konto	register.html
	`POST /register`	Opretter en ny bruger	Sendes til /login. Fejl: register.html med besked
	`POST /updateBalance`	Sætter penge ind på saldoen eller fjerner alle pengene	Sendes til /
	`GET /orderoverview`	Viser kundens tidligere ordrer	orderoverview.html
	`POST /deleteorder`	Sletter en ordre og dens ordrelinjer	Sendes til /orderoverview
BestilController	`GET /bestil`	Viser bestillingssiden med bunde og toppings, og kurven hvis den findes	bestilling.html. Uden login: sendes til /login
	`POST /bestil`	Opretter ordren, finder priserne i databasen og gemmer en ordrelinje for hver række	bestilling.html med oversigt til højre
	`POST /bestil/redigere`	Åbner formularen igen, så kunden kan ændre kurven	bestilling.html i redigeringstilstand
BetalController	`POST /betal`	Viser ordren og prisen og spørger "Er du sikker?"	betaling.html
	`POST /betal/bekraeft`	Tjekker saldoen. Er den høj nok, markeres ordren som betalt, og beløbet trækkes	betalingcomplete.html. For lav saldo: betaling.html med rød besked
AdminController tjekker at brugeren er admin	`GET /admin`	Sender admin videre til ordrelisten	Sendes til /admin/ordrer
	`GET /admin/kunder`	Viser alle kunder	adminkunde.html
	`GET /admin/kunder/{id}`	Viser én kunde og kundens ordrer	kundedetaljer.html
	`GET /admin/ordrer`	Viser alle ordrer	adminordre.html
	`POST /admin/ordrer/slet`	Sletter en kundes ordre	Sendes til /admin/ordrer

8 Test

Vi har testet systemet på to måder: med automatiske integrationstests af databaselaget og med manuelle tests af hele systemet i browseren.

Integrationstest

Vi har skrevet JUnit-tests til `BestilMapper`, `UserMapper` og `BetalMapper`. Testene kører mod en rigtig PostgreSQL-database i et separat schema, `test`, så de rigtige data aldrig bliver ændret. Alle testklasser arver fra en fælles basisklasse, `DatabaseTestBefore`, så opsætningen kun står ét sted:

- `@BeforeAll` kører én gang pr. testklasse og opretter forbindelsespuljen og mapperne.
- `@BeforeEach` kører før hver test. Den sletter data i rækkefølgen orderlines, orders, bottoms, toppings og users (på grund af fremmednøgler), nulstiller id'erne til 1 og indsætter faste testdata: 3 brugere, 3 bunde, 3 toppings, 2 ordrer og 2 ordrelinjer.

Fordi data nulstilles før hver test, er testene uafhængige af hinanden, og vi kender id'erne på forhånd. Vi tester blandt andet:

- at bunde og toppings kan hentes,
- at en ordre og en ordrelinje kan oprettes,
- at ordrens pris og status kan opdateres,
- at en bruger kan logge ind, og at forkert kode eller ukendt email giver en fejl,
- at en ordrelinje kan slettes.

Hvornår har vi testet?

Mapper-testene skrev vi, efter databaselaget var på plads. Siderne har vi testet manuelt i browseren undervejs, mens vi byggede dem: Vi klikkede flowet igennem fra login til betaling og rettede fejl ud fra beskederne i konsollen.

Manuelle testscenarier

Controllerne, HTML-skabelonerne og JavaScript til bestillingssiden har vi testet manuelt ved at gå igennem scenarierne nedenfor i browseren.

Nr.	Scenarie	Forventet resultat
1	Opret en konto med en ny e-mail	Kontoen oprettes, og man sendes til loginsiden
2	Opret en konto med en e-mail, der allerede findes	Beskeden "Emailen findes allerede"
3	Opret en konto med to forskellige adgangskoder	Beskeden "De to adgangskoder er ikke ens", og kontoen oprettes ikke
4	Log ind med forkert adgangskode	Beskeden "Forkert email eller adgangskode"

Nr.	Scenarie	Forventet resultat
5	Tryk på Bestil uden at være logget ind	Man sendes til loginsiden
6	Læg to forskellige cupcakes i kurven	Oversigten til højre viser hver linje og den samlede pris
7	Tryk på Redigere i oversigten	Formularen åbnes igen, så kurven kan ændres
8	Betal med nok saldo	Ordren bliver "Paid", saldoen trækkes, og siden "Betaling gennemført" vises
9	Betal med for lav saldo	Beskeden "utilstrækkelig saldo" vises, og ordren forbliver "unpaid"
10	Log ud og ind igen med en ubetalt ordre	Ordren står i ordrehistorikken med "Betal nu" og "Slet ordre"
11	Log ind som admin	Man ser alle kunder og alle ordrer
12	Åbn en admin-side som kunde	Man sendes til loginsiden
13	Slet en ordre som admin	Ordren forsvinder fra listen

9 Status på implementation

Alle 8 user stories er implementeret. Tre acceptkriterier er kun delvist opfyldt:

- **US-1:** Efter betaling vises siden "Betaling gennemført". Den viser hverken ordrenummer eller at ordren kan hentes i butikken.
- **US-3:** Når kurven er tom, vises der ingen besked. Kurven står bare tom.
- **US-4:** Emailen står ikke i topmenuen på siden med ordrehistorik.

Alle andre acceptkriterier i US-1 til US-8 er opfyldt og testet manuelt.

Kendte fejl og mangler

- Nogle POST-routes (`/betal` , `/betal/bekraeft` , `/bestil/redigere` , `/updateBalance`) tjekker ikke, om brugeren er logget ind. De kan kun kaldes fra knapper på siderne, men de burde stadig have et login-tjek.
- Adgangskoder gemmes i klar tekst i databasen. I et rigtigt system skulle de hashes, fx med BCrypt.
- Ved betaling opdateres ordrens status og kundens saldo i to separate SQL-kald uden en transaktion. Går noget galt midt imellem, kan data blive forkert.
- Vi har kun unit tests af mapperne, ikke af controllere og services.

10 Proces

Planlægning og værktøjer

- Vi har planlagt arbejdet med GitHub Projects.
- Hver user story (US-1 til US-8) blev oprettet som et issue på boardet og delt op i mindre tasks.
- Boardet havde kolonnerne Todo, In Progress og Done.
- Når nogen gik i gang med en opgave, flyttede de den til In Progress, så alle i gruppen kunne se, hvem der arbejdede på hvad.

Vi startede med at lave mockups i Figma, en domænemodel og et ER-diagram, inden vi begyndte at kode. Det gav os et fælles billede af, hvordan siderne og databasen skulle se ud.

Estimering

- Vi estimerede vores user stories med planning poker og story points (1, 2, 3, 5, 8, 13).
- Hver person vurderede først selv, hvor stor en user story var, og så viste alle deres tal på samme tid.
- Det gjorde vi, så ingen blev påvirket af de andres tal.

Som regel lå vores tal tæt på hinanden. Ofte var vi kun ét trin fra hinanden, fx hvis én sagde 3 og en anden 5. Så snakkede vi kort om, hvorfor vi havde valgt forskelligt, og blev enige om et tal. Det viste, at vi havde en ret ens forståelse af opgaverne. Når vi var uenige, kom vi ofte frem til ting, som én af os havde overset.

Git-strategi

- Vi har arbejdet med feature branches.
- Hver user story fik sin egen branch, fx `Login-US4`, `US-6-Administrator` og `US-5-Admin-ordrer`.
- Når en user story var færdig, lavede vi en pull request og merged den ind i `main`. På den måde var `main` altid en version, der virkede.
- Mindre rettelser i slutningen af projektet blev committet direkte på `main`.

Kodestandarder

- Klasser og metoder er mest skrevet på engelsk, fx `UserMapper` og `getOrdersByUserId`. Nogle navne er på dansk, fx `BestilController` og `BetalMapper`, fordi de følger siderne i webshoppen.
- Koden er delt i lag: controllers, services, mappers og entities/DTO'er.
- Al SQL ligger i mapperne og bruger `PreparedStatement`.
- Fejl fra databasen bliver lavet om til en `DatabaseException` med en forståelig besked.

Definition of Done

En user story var færdig, når:

1. alle acceptkriterier virkede i browseren
2. koden var testet manuelt
3. koden var merged ind i main via en pull request
4. opgaven var flyttet til Done på boardet.

Roller

- **Scrum Master:** Daud. Holdt styr på boardet og på, at vi fik holdt korte møder.
- **Product Owner:** Cezary. Prioriterede user stories og tjekkede, at de passede med Olsker Cupcakes' ønsker.
- **Tech Lead:** Feyzullah. Satte projektet og databasen op og hjalp med tekniske problemer.

I praksis var rollerne ikke helt faste. Når vi sad sammen, hjalp vi hinanden på tværs. Fx tog Feyzullah nogle gange rollen som Scrum Master og satte tidspunktet for vores møder.

Hvad der gik godt i vores projekt

- Lagdelingen gjorde det nemt at finde rundt i koden og at arbejde på forskellige dele samtidig.
- Feature branches betød, at vi sjældent ødelagde hinandens kode.

- Mockups i Figma gjorde det tydeligt, hvad hver side skulle indeholde.

Hvad der ikke gik så godt

- Tests blev skrevet sent i vores projekt.
- Vi oplevede også, at nogle metoder blev lavet to gange i forskellige mappers, fordi vi ikke havde aftalt, hvor tingene skulle ligge.
- Topmenuen blev kopieret ind i mange HTML-filer, så den skulle rettes flere steder.

Det vi kan gøre anderledes næste gang

- Lave en fælles topmenu som Thymeleaf-fragment fra starten.
- Skrive tests løbende, imens vi koder.
- Aftale navne på mappers og metoder tidligt.
- Lægge login-tjek på alle routes.
- Lave klassenavne på engelsk i stedet for dansk.